Experiment 15: BPTT Flow and Gradient Analysis

Question

Problem Statement: A stock-analytics firm finds its RNN ignores price movements from more than a few days ago. The team suspects the gradients are vanishing. Daily closing prices of a stock index over 3 years, with sequence lengths of 10, 50 and 100 days. The objective is to trace backpropagation through time and measure gradient magnitude at each time step. Use TensorFlow, Keras, NumPy, Matplotlib.

Tasks:

1. Build a SimpleRNN and use tf.GradientTape to compute gradients of the loss.
2. Record the gradient norm with respect to the hidden state at each time step.
3. Plot gradient norm against time step for each sequence length.
4. Compare tanh and ReLU activations.
5. Explain vanishing and exploding gradients from the plots.

Code

```
"""Experiment 15: BPTT Flow and Gradient Analysis (vanishing / exploding gradients)"""
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

np.random.seed(42); tf.random.set_seed(42)

# ---------------- Data: 3 years of daily closing prices (synthetic geometric random walk) ----------------
n_days = 3 * 252
returns = np.random.normal(0.0004, 0.012, n_days)
price = 1000 * np.exp(np.cumsum(returns))
price = (price - price.mean()) / price.std()          # standardise
# To use real data:  price = pd.read_csv("index.csv")["Close"].values  (then standardise)

def make_windows(series, T, n=32):
    idx = np.random.randint(0, len(series) - T - 1, n)
    X = np.stack([series[i:i + T] for i in idx])[..., None].astype("float32")
    y = np.array([series[i + T] for i in idx], dtype="float32").reshape(-1, 1)
    return tf.constant(X), tf.constant(y)

# ---------------- Tasks 1 & 2: BPTT with tf.GradientTape ----------------
def gradient_norms(T, activation="tanh", units=32, rec_scale=1.0, seed=0):
    """Return ||dL/dh_t|| for t = 1..T (averaged over a mini-batch)."""
    tf.random.set_seed(seed); np.random.seed(seed)
    cell = layers.SimpleRNNCell(units, activation=activation)     # the recurrent cell of a SimpleRNN
    head = layers.Dense(1)
    X, y = make_windows(price, T)
    cell.build((None, 1)); head.build((None, units))
    cell.recurrent_kernel.assign(cell.recurrent_kernel * rec_scale)  # optional scaling of W_hh
    with tf.GradientTape() as tape:
        h = tf.zeros((X.shape[0], units))
        hs = []
        for t in range(T):                                        # unrolled forward pass (time loop)
            h, _ = cell(X[:, t, :], [h])
            hs.append(h)
        loss = tf.reduce_mean(tf.square(head(h) - y))             # loss depends on the LAST hidden state
    grads = tape.gradient(loss, hs)                               # dL/dh_t for every time step (BPTT)
    return np.array([tf.reduce_mean(tf.norm(g, axis=1)).numpy() for g in grads])

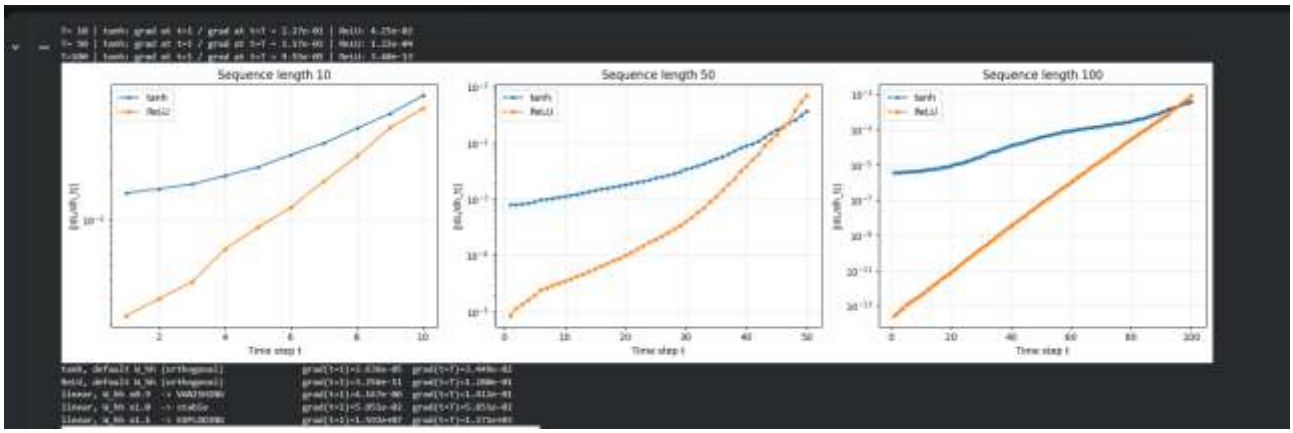
# ---------------- Task 3 & 4: gradient norm vs time step, tanh vs ReLU ----------------
fig, ax = plt.subplots(1, 3, figsize=(17, 4.5))
for a, T in zip(ax, [10, 50, 100]):
    g_tanh = gradient_norms(T, "tanh")
    g_relu = gradient_norms(T, "relu")
    a.plot(range(1, T + 1), g_tanh, "o-", ms=3, label="tanh")
    a.plot(range(1, T + 1), g_relu, "s-", ms=3, label="ReLU")
    a.set_yscale("log"); a.set_xlabel("Time step t"); a.set_ylabel("||dL/dh_t||")
    a.set_title(f"Sequence length {T}"); a.legend(); a.grid(alpha=0.3)
```

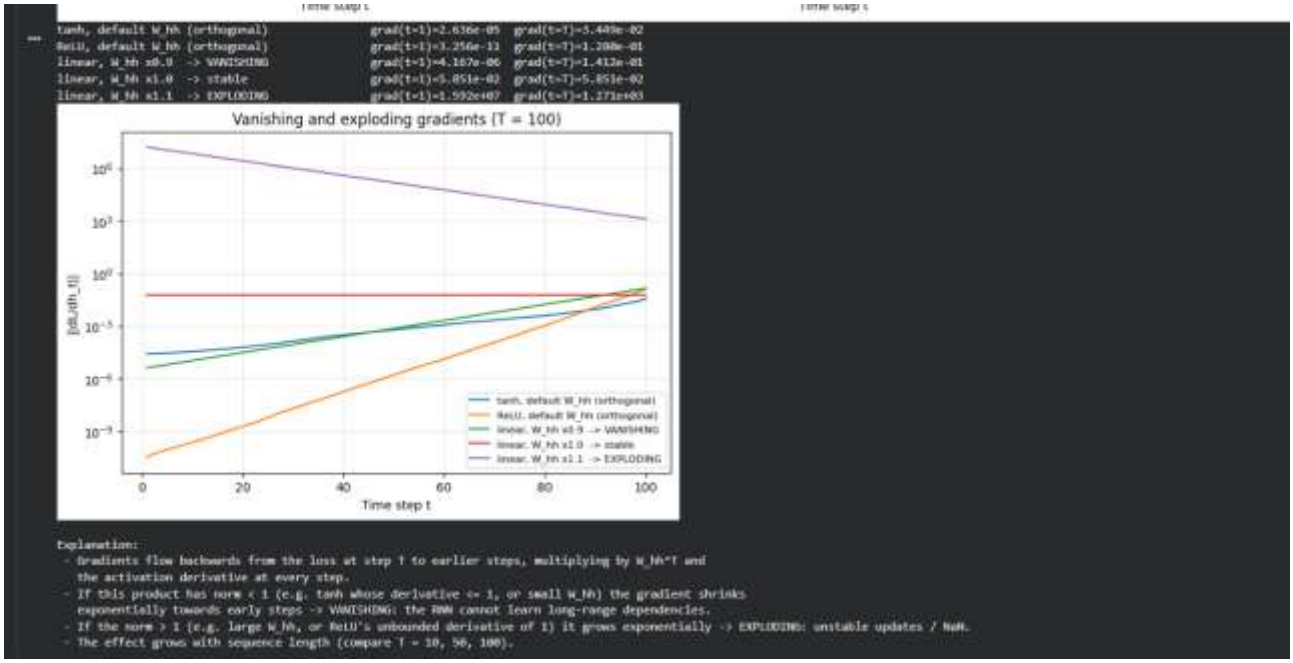
```
    print(f"T={T:3d} | tanh: grad at t=1 / grad at t=T = {g_tanh[0]/g_tanh[-1]:.2e} | "
          f"ReLU: {g_relu[0]/g_relu[-1]:.2e}")
plt.tight_layout(); plt.show()


# ---------------- Task 5: vanishing vs exploding ----------------
# A linear activation isolates the effect of W_hh: the gradient is multiplied by ~(scale)^k after k steps.
T = 100
plt.figure(figsize=(8, 5))
for act, scale, label in [("tanh", 1.0, "tanh, default W_hh (orthogonal)"),
                          ("relu", 1.0, "ReLU, default W_hh (orthogonal)"),
                          ("linear", 0.9, "linear, W_hh x0.9  -> VANISHING"),
                          ("linear", 1.0, "linear, W_hh x1.0  -> stable"),
                          ("linear", 1.1, "linear, W_hh x1.1  -> EXPLODING")]:
    g = gradient_norms(T, act, rec_scale=scale)
    plt.plot(range(1, T + 1), g, label=label)
    print(f"{label:45s} grad(t=1)={g[0]:.3e}  grad(t=T)={g[-1]:.3e}")
plt.yscale("log"); plt.xlabel("Time step t"); plt.ylabel("||dL/dh_t||")
plt.title("Vanishing and exploding gradients (T = 100)"); plt.legend(fontsize=8); plt.grid(alpha=0.3);
plt.show()

print("""
Explanation:
 - Gradients flow backwards from the loss at step T to earlier steps, multiplying by W_hh^T and
   the activation derivative at every step.
 - If this product has norm < 1 (e.g. tanh whose derivative <= 1, or small W_hh) the gradient shrinks
   exponentially towards early steps -> VANISHING: the RNN cannot learn long-range dependencies.
 - If the norm > 1 (e.g. large W_hh, or ReLU's unbounded derivative of 1) it grows exponentially ->
EXPLODING: unstable updates / NaN.
 - The effect grows with sequence length (compare T = 10, 50, 100).""")
```

Output


Sequence length 10
Sequence length 50
Sequence length 100
tanh
ReLU
Time step t


tanh, default W_hh (orthogonal) grad(t=1)=2.636e-05 grad(t=T)=5.449e-02
ReLU, default W_hh (orthogonal) grad(t=1)=3.256e-13 grad(t=T)=1.208e-01
linear, W_hh x0.9 -> VANISHING grad(t=1)=4.167e-06 grad(t=T)=1.413e-01
linear, W_hh x1.0 -> stable grad(t=1)=5.851e-02 grad(t=T)=5.851e-02
linear, W_hh x1.1 -> EXPLODING grad(t=1)=1.592e+07 grad(t=T)=1.271e+03
Vanishing and exploding gradients (T = 100)
|dL/dh_t|
Time step t
tanh, default W_hh (orthogonal)
ReLU, default W_hh (orthogonal)
linear, W_hh x0.9 -> VANISHING
linear, W_hh x1.0 -> stable
linear, W_hh x1.1 -> EXPLODING
Explanation:
- Gradients flow backwards from the loss at step t to earlier steps, multiplying by W_hh^T and
the activation derivative at every step.
- If this product has norm < 1 (e.g. tanh whose derivative <= 1, or small W_hh) the gradient shrinks
exponentially towards early steps -> VANISHING: the RNN cannot learn long-range dependencies.
- If the norm > 1 (e.g. large W_hh, or ReLU's unbounded derivative of 1) it grows exponentially -> EXPLODING: unstable updates / NaN.
- The effect grows with sequence length (compare T = 10, 50, 100).